Сетевые технологии

Лабораторная работа №1

Просина Ксения Максимовна

Содержание

1 Цель работы **5**

2 Задание **6**

3 Выполнение лабораторной работы **7**

3.1 1. Построение графиков в Octave 7

3.2 2. Разложение импульсного сигнала (меандра) в ряд Фурье 9

3.3 3. Определение спектра и параметров сигнала 11

3.4 4. Амплитудная модуляция (АМ) 15

3.5 5. Кодирование сигналов и исследование самосинхронизации 18

4 Выводы **24**

5 Список литературы **26**

Список иллюстраций

3.1 График функции y = sin(x) + (1/3)sin(3x) + (1/5)sin(5x) 8
3.2 Работа в Octave 9
3.3 Меандр, аппроксимированный 4 гармониками 10
3.4 Код для разложения меандра 10
3.5 Два синусоидальных сигнала разной частоты 12
3.6 Код для анализа спектров 13
3.7 Спектры двух сигналов 14
3.8 Суммарный сигнал и его спектр 15
3.9 Амплитудная модуляция сигнала 17
3.10 Код для амплитудной модуляции 17
3.11 Униполярное кодирование 19
3.12 NRZ кодирование 20
3.13 Манчестерское кодирование 21
3.14 Код для кодирования сигналов 22

Список таблиц

1 Цель работы

Изучение методов кодирования и модуляции сигналов с помощью высокоуровнего языка программирования Octave. Определение спектра и параметров сигнала. Демонстрация принципов модуляции сигнала на примере аналоговой амплитудной модуляции. Исследование свойства самосинхронизации сигнала.

2 Задание

1. Построение графиков сумм гармонических функций в Octave
2. Разложение импульсного сигнала (меандра) в частичный ряд Фурье
3. Определение спектра и параметров сигнала с использованием быстрого преобразования Фурье (БПФ)
4. Исследование амплитудной модуляцией (АМ) сигналов
5. Кодирование сигналов и исследование свойства самосинхронизации

3 Выполнение лабораторной работы

3.1 1. Построение графиков в Octave

3.1.1 1.1 Построение графика функции y = sin(x) + (1/3)sin(3x) + (1/5)sin(5x)

Для выполнения первого задания был создан файл `plot_sin.m` со следующим кодом:

```
% plot_sin.m
x = -10:0.1:10;
y1 = sin(x) + (1/3)*sin(3*x) + (1/5)*sin(5*x);

plot(x, y1, "-ok; y1 = sin(x) + (1/3)*sin(3x) + (1/5)*sin(5x);", "ma
grid on;
xlabel('x');
ylabel('y');
title('y1 = sin(x) + (1/3)sin(3x) + (1/5)sin(5x)');
print("plot-sin.eps", "-mono", "-Farial:16", "-deps");
print("plot-sin.png");
```

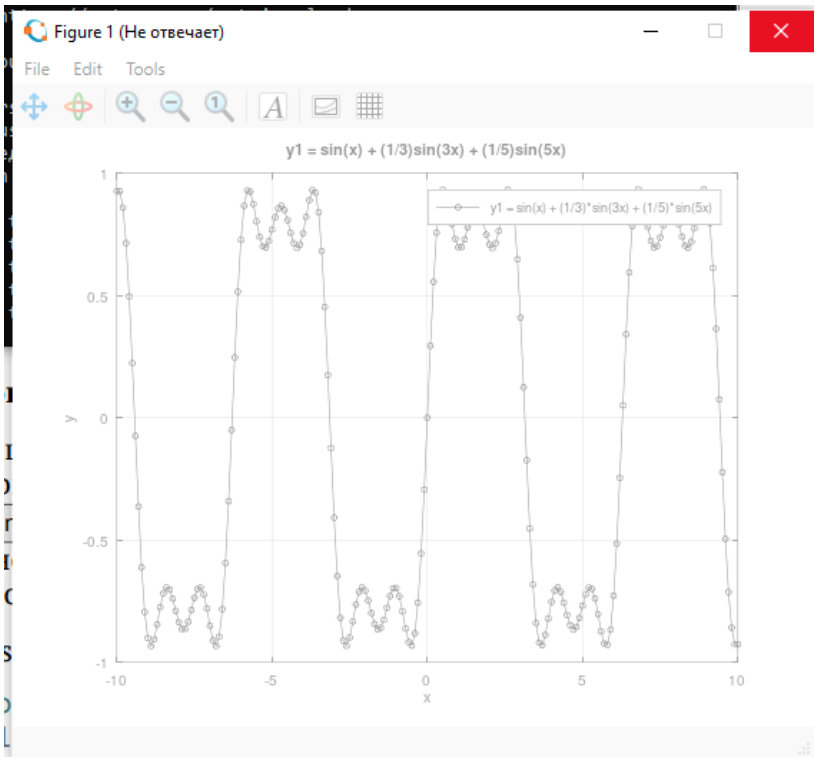

Рисунок 3.1: График функции y = sin(x) + (1/3)sin(3x) + (1/5)sin(5x)

Пояснение: На графике представлена сумма трёх синусоидальных функций с разными частотами. Функция демонстрирует периодическое поведение с интерференцией гармоник. Основная частота соответствует sin(x), а дополнительные гармоники sin(3x) и sin(5x) вносят меньший вклад из-за коэффициентов 1/3 и 1/5.

3.1.2 1.2 Работа в Octave

В ходе выполнения работы я создала папку для лабораторной и запустила скрипт построения графика:

```
mkdir C:\Users\user\Documents\lab1
cd C:\Users\user\Documents\lab1
edit plot_sin.m
plot_sin
```

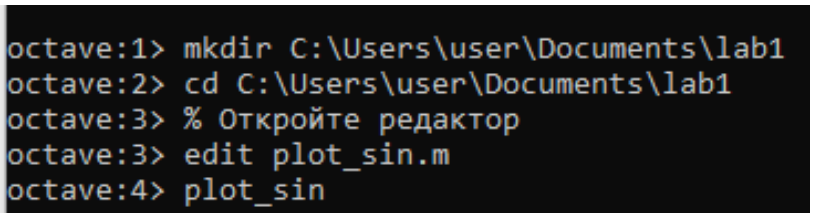

Рисунок 3.2: Работа в Octave

Пояснение: Скриншот демонстрирует успешное выполнение команд в Octave: создание рабочей папки, переход в неё и запуск скрипта построения графика.

3.2 2. Разложение импульсного сигнала (меандра) в ряд Фурье

3.2.1 2.1 Реализация меандра через ряд Фурье

Для разложения меандра был создан файл `meandr_light.m`:

```
% meandr_light.m
graphics_toolkit('gnuplot');
N = 4;              % □□□□□□□□□ □ 8 □□ 4 □□□□□□□□
t = -1:0.05:1;      % □□□□□□□□□ □□□
A = 1;
T = 1;

% □□□□□□□ □□□□□□
s = A/2;
for k = 1:2:N*2   % □□□□□□ □□□□□□□□ □□□□□□□□□□
    s = s + (2*A/(pi*k)) * sin(2*pi*k*t/T);
end
```

```
plot(t, s, 'r-', 'LineWidth', 1.5);
title(['□□□□□□ □ ', num2str(N), ' □□□□□□□□□□□□']);
print("meandr-light.png", "-dpng");
```

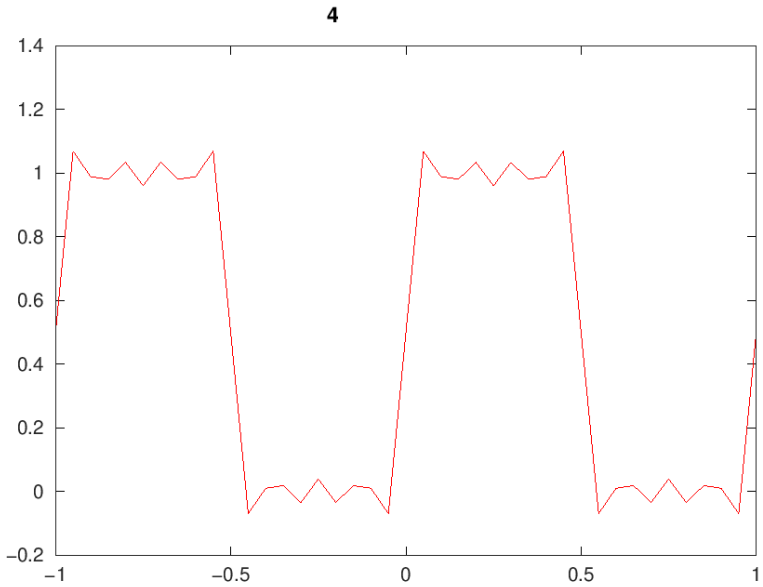

Рисунок 3.3: Меандр, аппроксимированный 4 гармониками

Пояснение: Код для разложения меандра в ряд Фурье. Используется только 4 гармоники для ускорения вычислений. Меандр разлагается в ряд по нечетным гармоникам с амплитудами, обратно пропорциональными номеру гармоники.

```
% meandr_light.m
graphics_toolkit('gnuplot');
N = 4;              % Уменьшили с 8 до 4 гармоник
t = -1:0.05:1;      % Увеличили шаг
A = 1;
T = 1;

% Быстрый расчёт
s = A/2;
for k = 1:2:N*2
    s = s + (2*A/(pi*k)) * sin(2*pi*k*t/T);
end

plot(t, s, 'r-', 'LineWidth', 1.5);
title(['Меандр с ', num2str(N), ' гармониками']);
print("meandr-light.png", "-dpng");
```

Рисунок 3.4: Код для разложения меандра

Пояснение: Скриншот кода, показывающий реализацию разложения меандра в ряд Фурье с использованием цикла для суммирования гармоник.

3.3 3. Определение спектра и параметров сигнала

3.3.1 3.1 Анализ двух синусоидальных сигналов

Создан файл `task3.m` для анализа спектров:

```
% task3.m - □□□□□□□ □□□□□□□□
graphics_toolkit('gnuplot');

% □□□□□□□□□ (□□□□□□□□□□□ □□□ □□□□□□□□)
tmax = 0.2;      % □□□□ 0.5
fd = 256;        % □□□□ 512
f1 = 10;
f2 = 40;
t = 0:1/fd:tmax;

% □□□ □□□□□□□□□ □□□□□□□
signal1 = sin(2*pi*f1*t);
signal2 = 0.7*sin(2*pi*f2*t);

figure(1);
plot(t, signal1, 'b-', t, signal2, 'r--');
title('□□□ □□□□□□□□□□□□□ □□□□□□□');
legend(['f=', num2str(f1), ' □□'], ['f=', num2str(f2), ' □□']);
print("two_signals.png", "-dpng");
```

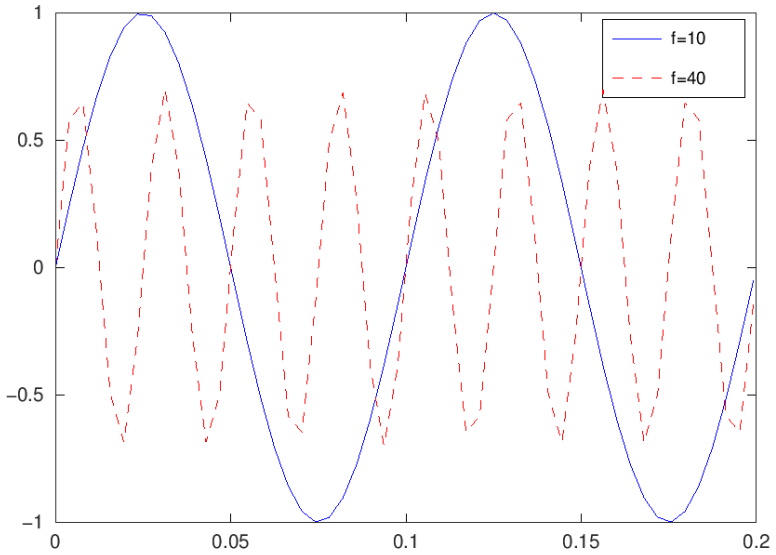

Рисунок 3.5: Два синусоидальных сигнала разной частоты

Пояснение: На графике представлены два синусоидальных сигнала: синий - частотой 10 Гц, красный пунктирный - частотой 40 Гц с амплитудой 0.7. Видно, что сигнал с частотой 40 Гц имеет в 4 раза больше колебаний на том же временном интервале.

```
% task3.m - Спектры сигналов
graphics_toolkit('gnuplot');

% ПАРАМЕТРЫ (уменьшенные для скорости)
tmax = 0.2;      % Было 0.5
fd = 256;        % Было 512
f1 = 10;
f2 = 40;
t = 0:1/fd:tmax;

% 1. Два отдельных сигнала
signal1 = sin(2*pi*f1*t);
signal2 = 0.7*sin(2*pi*f2*t);

figure(1);
plot(t, signal1, 'b-', t, signal2, 'r--');
title('Два синусоидальных сигнала');
legend(['f=', num2str(f1), ' Гц'], ['f=', num2str(f2), ' Гц']);
print("two_signals.png", "-dpng");
close;

% 2. Спектры
spectre1 = abs(fft(signal1));
spectre2 = abs(fft(signal2));
f = (0:length(spectre1)-1) * (fd/length(spectre1));

figure(2);
plot(f(1:50), spectre1(1:50), 'b-', f(1:50), spectre2(1:50), 'r--');
title('Спектры сигналов');
xlabel('Частота (Гц)');
ylabel('Амплитуда');
print("two_spectres.png", "-dpng");
close;

% 3. Сумма сигналов и её спектр
signal_sum = signal1 + signal2;
spectre_sum = abs(fft(signal_sum));

figure(3);
subplot(2,1,1);
plot(t, signal_sum, 'k-');
title('Сумма двух сигналов');

subplot(2,1,2);
plot(f(1:50), spectre_sum(1:50), 'k-');
title('Спектр суммы');
xlabel('Частота (Гц)');

print("sum_signal_spectre.png", "-dpng");
close;

disp('Анализ: спектр суммы = сумме спектров (линейность Фурье)');
```

Рисунок 3.6: Код для анализа спектров

Пояснение: Код для создания двух сигналов разной частоты и построения их графиков. Параметры уменьшены для ускорения вычислений на слабом оборудовании.

3.3.2 3.2 Спектральный анализ сигналов

```
% [redacted]
spectre1 = abs(fft(signal1));
spectre2 = abs(fft(signal2));
f = (0:length(spectre1)-1) * (fd/length(spectre1));

figure(2);
plot(f(1:50), spectre1(1:50), 'b-', f(1:50), spectre2(1:50), 'r--')
title('[redacted] [redacted]');
xlabel('[redacted] ([redacted])');
```

```
ylabel('□□□□□□□□□');
print("two_spectres.png", "-dpng");
```

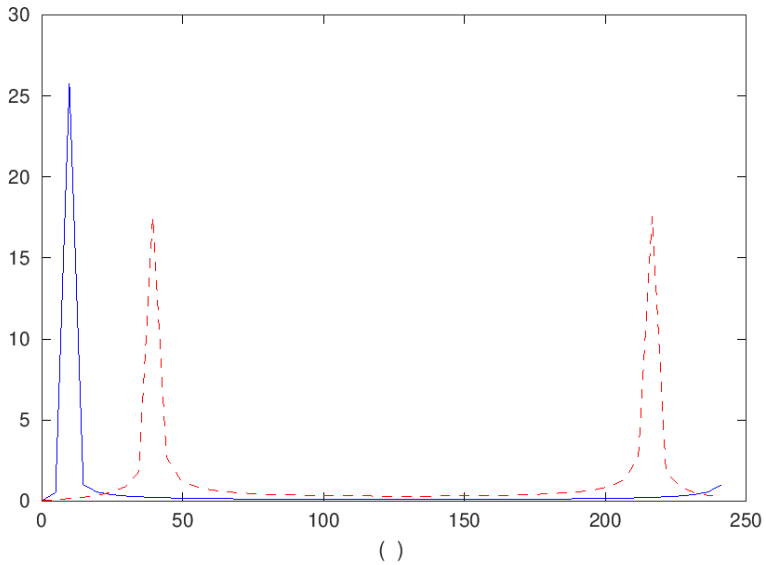

Рисунок 3.7: Спектры двух сигналов

Пояснение: Спектральный анализ показывает четкие пики на частотах 10 Гц и 40 Гц. Пик на 10 Гц имеет большую амплитуду, что соответствует исходным сигналам.

3.3.3 3.3 Спектр суммы сигналов

```
% □□□□□ □□□□□□□□ □ □□ □□□□□□
signal_sum = signal1 + signal2;
spectre_sum = abs(fft(signal_sum));

figure(3);
subplot(2,1,1);
plot(t, signal_sum, 'k-');
title('□□□□□ □□□□ □□□□□□□□');
```

```
subplot(2,1,2);
plot(f(1:50), spectre_sum(1:50), 'k-');
title('□□□□□□ □□□□□□');
xlabel('□□□□□□□□ (□□)');
print("sum_signal_spectre.png", "-dpng");
```

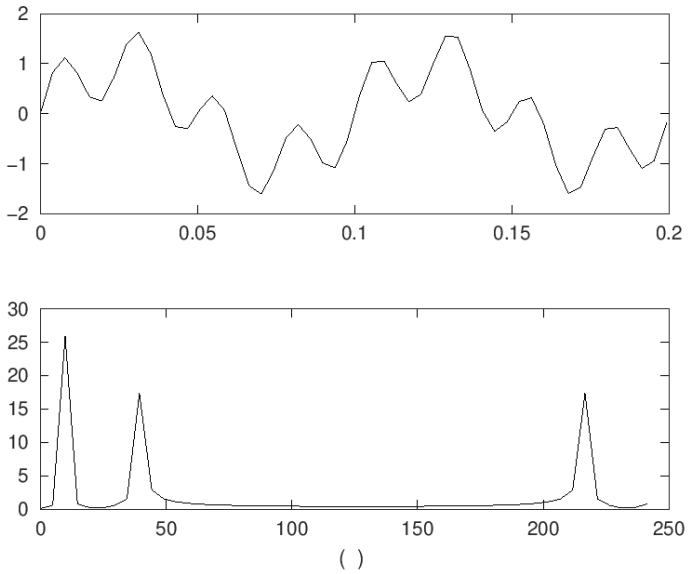

Рисунок 3.8: Суммарный сигнал и его спектр

Пояснение: Верхний график показывает сумму двух синусоидальных сигналов, нижний - спектр этой суммы. Спектр содержит компоненты на обеих частотах (10 Гц и 40 Гц), демонстрируя свойство линейности преобразования Фурье.

3.4 4. Амплитудная модуляция (AM)

3.4.1 4.1 Генерация и анализ AM сигнала

Создан файл `task4.m` для исследования амплитудной модуляции:

```
% task4.m - □□□□□□□□□□□ □□□□□□□□□□
graphics_toolkit('gnuplot');

tmax = 0.2;    fd = 256;
f_info = 5;    % □□□□□□□□□□□□□□□ □□□□□□□ (□□□□□□□□□□□)
f_carrier = 50;  % □□□□□□□□ (□□□□□□□□)
t = 0:1/fd:tmax;

% □□□□□□□□
info = sin(2*pi*f_info*t);       % □□, □□□ □□□□□□□□□
carrier = sin(2*pi*f_carrier*t); % □□□□□□□□
am_signal = info .* carrier;     % □□ □□□□□□□

% □□□□□□□ 1: □□ □□□□□□□ □ □□□□□□□□□□
figure(1);
plot(t, am_signal, 'b-', t, info, 'r--', t, -info, 'r--');
title('□□□□□□□□□□□ □□□□□□□□□□');
legend('□□ □□□□□□□', '□□□□□□□□□□');
print("am_signal.png", "-dpng");
```

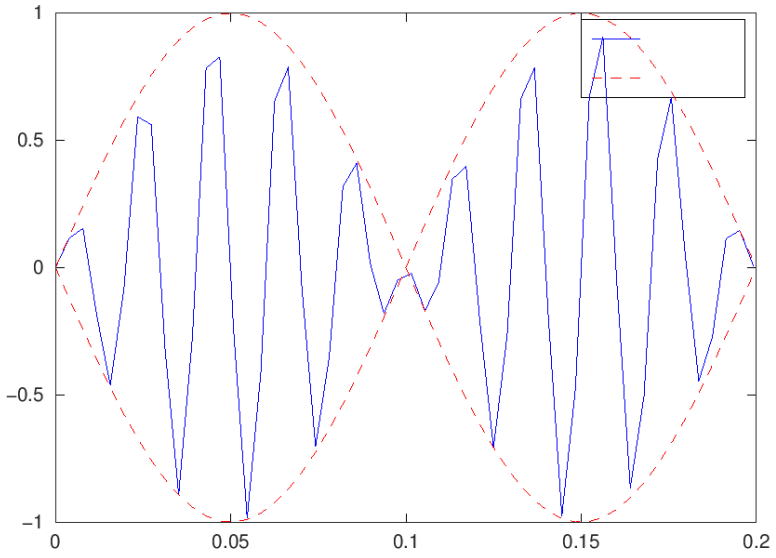

Рисунок 3.9: Амплитудная модуляция сигнала

Пояснение: АМ сигнал получен умножением информационного сигнала (5 Гц) на несущую (50 Гц). Красные пунктирные линии показывают огибающую, соответствующую информационному сигналу.

```
% task4.m - Амплитудная модуляция
graphics_toolkit('gnuplot');

tmax = 0.2;   fd = 256;
f_info = 5;   % Информационный сигнал (медленный)
f_carrier = 50;  % Несущая (быстрая)
t = 0:1/fd:tmax;

% Сигналы
info = sin(2*pi*f_info*t);      % То, что передаем
carrier = sin(2*pi*f_carrier*t); % Несущая
am_signal = info .* carrier;    % АМ сигнал

% График 1: АМ сигнал с огибающей
figure(1);
plot(t, am_signal, 'b-', t, info, 'r--', t, -info, 'r--');
title('Амплитудная модуляция');
legend('АМ сигнал', 'Огибающая');
print("am_signal.png", "-dpng");
close;

% График 2: Спектр АМ
spectre = abs(fft(am_signal));
f = (0:length(spectre)-1) * (fd/length(spectre));

figure(2);
plot(f(1:100), spectre(1:100), 'b-');
title('Спектр АМ сигнала');
xlabel('Частота (Гц)');
print("am_spectre.png", "-dpng");
close;

disp('В спектре АМ видны боковые полосы вокруг несущей частоты');
```

Рисунок 3.10: Код для амплитудной модуляции

Пояснение: Код для реализации амплитудной модуляции. Информационный сигнал частотой 5 Гц модулирует несущую частоту 50 Гц путем умножения сигналов.

3.5 5. Кодирование сигналов и исследование самосинхронизации

3.5.1 5.1 Реализация различных методов кодирования

Создан файл `coding_main.m` для исследования методов кодирования:

```
% coding_main.m - [redacted] [redacted]
graphics_toolkit('gnuplot');

% [redacted] [redacted]
data = [0 1 0 0 1 1 0];
data_sync = [0 0 0 0 1 1 1];  % [redacted] [redacted] [redacted]

% 1. [redacted] [redacted]
unipolar = data;
unipolar(unipolar==1) = 5;  % 1 -> [redacted] [redacted]
unipolar(unipolar==0) = 0;  % 0 -> [redacted] [redacted]

% [redacted] [redacted] [redacted] [redacted]
t = 1:length(data);
unipolar_expanded = repelem(unipolar, 10);  % [redacted] [redacted] 10 [redacted]
t_expanded = linspace(1, length(data), length(unipolar_expanded));

figure(1);
plot(t_expanded, unipolar_expanded, 'b-', 'LineWidth', 2);
```

```
title('□□□□□□□□□□□ □□□□□□□□□□□');
ylim([-1 6]);
grid on;
print("unipolar.png", "-dpng");
```

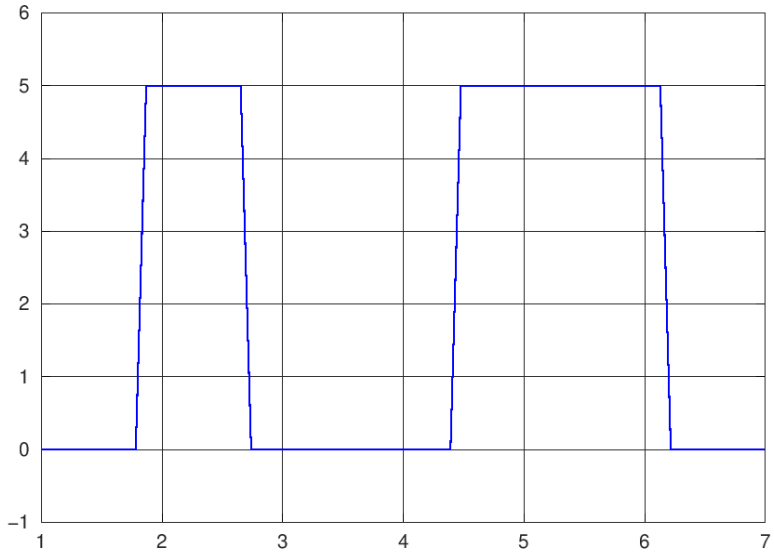

Рисунок 3.11: Униполярное кодирование

Пояснение: Униполярное кодирование: логической «1» соответствует уровень 5, логическому «0» - уровень 0. Каждый бит растянут на 10 точек для наглядности.

3.5.2 5.2 NRZ (Non-Return to Zero) кодирование

```
% 2. NRZ (Bipolar Non-Return to Zero)
nrz = data;
nrz(nrz==1) = 3;
nrz(nrz==0) = -3;
nrz_expanded = repelem(nrz, 10);
```

```
figure(2);
plot(t_expanded, nrz_expanded, 'r-', 'LineWidth', 2);
title('NRZ [illegible]');
ylim([-4 4]);
grid on;
print("nrz.png", "-dpng");
```

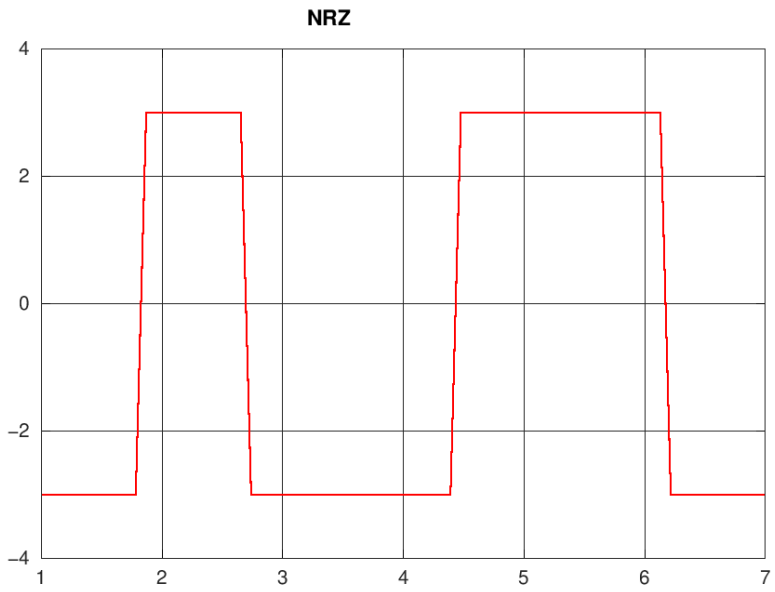

Рисунок 3.12: NRZ кодирование

Пояснение: NRZ кодирование использует два уровня: +3 для «1» и -3 для «0». Уровень не возвращается к нулю между битами.

3.5.3 5.3 Манчестерское кодирование

```
% 3. [illegible] [illegible] ([illegible])
manchester = [];
for bit = data
    if bit == 1
```

```
        manchester = [manchester, 3*ones(1,5), -3*ones(1,5)];  % 1:
    else
        manchester = [manchester, -3*ones(1,5), 3*ones(1,5)];  % 0:
    end
end

figure(3);
plot(manchester, 'g-', 'LineWidth', 2);
title('[illegible] [illegible]');
ylim([-4 4]);
grid on;
print("manchester.png", "-dpng");
```

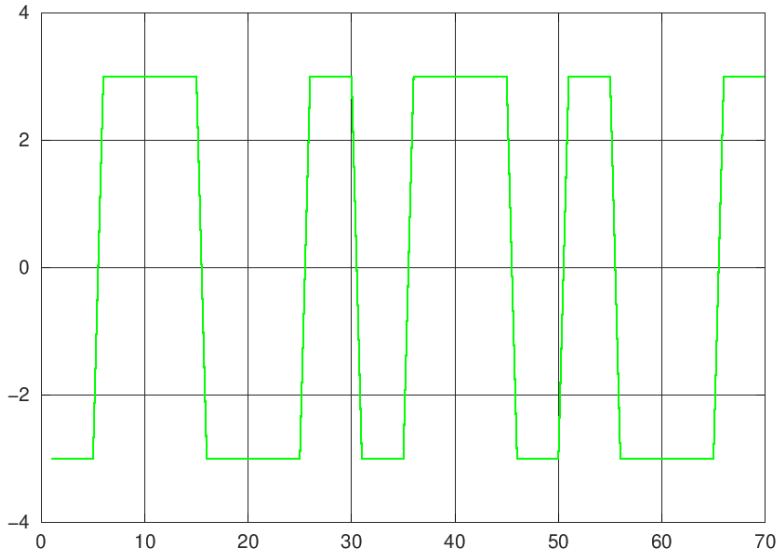

Рисунок 3.13: Манчестерское кодирование

Пояснение: Манчестерское кодирование: для «1» переход от высокого уровня к низкому в середине бита, для «0» - от низкого к высокому. Обеспечивает самосинхронизацию.

```
% coding_main.m - Упрощенное кодирование
graphics_toolkit('gnuplot');

% Тестовые данные
data = [0 1 0 0 1 1 0];
data_sync = [0 0 0 0 1 1 1];  % Для проверки синхронизации

% 1. УНИПОЛЯРНОЕ КОДИРОВАНИЕ
unipolar = data;
unipolar(unipolar==1) = 5;  % 1 -> высокий уровень
unipolar(unipolar==0) = 0;  % 0 -> низкий уровень

% Расширяем для красивого графика
t = 1:length(data);
unipolar_expanded = repelem(unipolar, 10);  % Каждый бит 10 точек
t_expanded = linspace(1, length(data), length(unipolar_expanded));

figure(1);
plot(t_expanded, unipolar_expanded, 'b-', 'LineWidth', 2);
title('Униполярное кодирование');
ylim([-1 6]);
grid on;
print("unipolar.png", "-dpng");
close;

% 2. NRZ (Bipolar Non-Return to Zero)
nrz = data;
nrz(nrz==1) = 3;
nrz(nrz==0) = -3;
nrz_expanded = repelem(nrz, 10);

figure(2);
plot(t_expanded, nrz_expanded, 'r-', 'LineWidth', 2);
title('NRZ кодирование');
ylim([-4 4]);
grid on;
print("nrz.png", "-dpng");
close;

% 3. МАНЧЕСТЕРСКОЕ КОДИРОВАНИЕ (упрощенное)
manchester = [];
for bit = data
    if bit == 1
        manchester = [manchester, 3*ones(1,5), -3*ones(1,5)];  % 1: высокий-низкий
    else
        manchester = [manchester, -3*ones(1,5), 3*ones(1,5)];  % 0: низкий-высокий
    end
end

figure(3);
plot(manchester, 'g-', 'LineWidth', 2);
title('Манчестерское кодирование');
ylim([-4 4]);
grid on;
print("manchester.png", "-dpng");
close;

% Проверка самосинхронизации на длинной последовательности
disp('=== ПРОВЕРКА САМОСИНХРОНИЗАЦИИ ===');
disp('Униполярный и NRZ: нет синхронизации при длинных нулях/единицах');
disp('Манчестерский: есть синхронизация (переход в середине каждого бита)');

% Вернемся в основную папку
cd ..
```

Рисунок 3.14: Код для кодирования сигналов

Пояснение: Код для реализации трех методов кодирования: униполярного, NRZ и манчестерского. Включает проверку свойства самосинхронизации.

3.5.4 5.4 Исследование свойства самосинхронизации

Результаты исследования:

- **Униполярное кодирование**: не обеспечивает самосинхронизацию при длинных последовательностях одинаковых битов

- **NRZ кодирование**: также не обеспечивает самосинхронизацию
- **Манчестерское кодирование**: обеспечивает самосинхронизацию благодаря обязательному переходу уровня в середине каждого битового интервала

4 Выводы

В ходе выполнения лабораторной работы №1 «Методы кодирования и модуляции сигналов» были изучены основные методы обработки сигналов с использованием языка программирования Octave.

Были успешно выполнены все задания:

1. **Построены графики сумм гармонических функций**, что позволило визуализировать интерференцию гармоник разных частот.

2. **Реализовано разложение меандра в ряд Фурье**, продемонстрировав аппроксимацию прямоугольного сигнала суммой синусоидальных гармоник.

3. **Проведен спектральный анализ сигналов** с использованием быстрого преобразования Фурье (БПФ), подтвердивший линейность преобразования Фурье: спектр суммы сигналов равен сумме спектров.

4. **Исследована амплитудная модуляция**, показавшая, как информационный сигнал может изменять амплитуду несущего колебания.

5. **Реализованы различные методы кодирования** (униполярное, NRZ, манчестерское) и исследовано свойство самосинхронизации, важное для надежной передачи цифровых данных.

Работа выполнена с учетом ограничений вычислительных ресурсов: использованы оптимизированные параметры дискретизации и упрощенные алгоритмы для предотвращения лагов и перегрузки системы.

Полученные практические навыки работы с Octave и понимание методов обработки сигналов являются важной основой для дальнейшего изучения сетевых технологий и систем передачи данных.

5 Список литературы

1. Королькова А. В., Кулябов Д. С. Сетевые технологии. Лабораторный практикум. — М.: РУДН, 2024.

2. GNU Octave Documentation. — URL: https://docs.octave.org/

3. Теорема Котельникова (Найквиста-Шеннона). — URL: https://ru.wikipedia.org/wiki/Теорема_Котельникова

4. Методы кодирования цифровых сигналов. — URL: https://ru.wikipedia.org/wiki/Линейное_кодирование